# Panduan Membuat Fitur — Backend

Manote · Java 21, Spring Boot 4, PostgreSQL · contoh acuan: fitur login

## 1. Gambaran alur

Setiap fitur backend mengikuti urutan lapisan yang sama. Request masuk dari kiri, data keluar dari kanan.

**Controller** → **Service** → **Repository** → **Database**

| Lapisan | Tugasnya | Contoh di fitur login |
|---|---|---|
| Controller | Menerima request HTTP, validasi bentuk input, memanggil service. | `AuthController` |
| Service | Logika bisnis. Di sinilah aturan fitur ditulis. | `AuthService`, `JwtService` |
| Repository | Query ke database lewat Spring Data JPA. | `UserRepository` |
| Entity | Cerminan satu tabel. | `User` |
| DTO | Bentuk JSON request dan response. Terpisah dari entity. | `LoginRequest`, `LoginResponse`, `UserResponse` |
| Migrasi | Perubahan skema database (Flyway). | `V1__create_users.sql` |

## 2. Peta file fitur login

```
backend/src/main/
├── java/com/hidarahlab/manote/
│   ├── auth/
│   │   ├── AuthController.java      POST /api/auth/login, GET /api/auth/me
│   │   ├── AuthService.java         cek password, buat response
│   │   ├── JwtService.java          membuat access token
│   │   └── dto/                     LoginRequest, LoginResponse, UserResponse
│   ├── user/
│   │   ├── User.java                entity tabel users
│   │   ├── UserRepository.java
│   │   └── UserType.java            USER, ADMIN
│   ├── common/                      ApiResponse, ErrorCode, ApiException, GlobalExceptionHandler
│   └── config/SecurityConfig.java   aturan endpoint, JWT, CORS
└── resources/
    ├── application.properties
    └── db/migration/                V1__create_users.sql, V2__seed_first_user.sql
```

## 3. Langkah membuat fitur baru

Contoh yang dipakai: fitur **edit profil** (mengubah nama lengkap).

### Langkah 1 — Tentukan kontrak API

Tulis dulu sebelum coding, karena web dan iOS akan mengikuti kontrak ini.

| Hal | Nilai |
|---|---|
| Method dan URL | `PUT /api/profile` |
| Butuh login? | Ya (kirim header `Authorization: Bearer <token>`) |
| Request | `{ "fullName": "Nama Baru" }` |
| Response 200 | `{ "userId", "email", "fullName", "userType" }` |
| Error | 400 jika nama kosong, 401 jika token tidak valid |

### Langkah 2 — Migrasi database (hanya jika skema berubah)

Mengubah nama lengkap tidak butuh kolom baru, jadi langkah ini dilewati. Kalau fitur butuh kolom baru, misalnya nomor telepon, buat file baru dengan nomor versi berikutnya:

```
-- src/main/resources/db/migration/V3__add_phone_to_users.sql
ALTER TABLE users ADD COLUMN phone VARCHAR(30);
```

> **Aturan Flyway:** jangan pernah mengedit file migrasi yang sudah pernah dijalankan. Selalu buat file baru (V3, V4, ...). Nama file memakai dua garis bawah setelah nomor versi.

### Langkah 3 — Entity dan Repository

Tambahkan field atau method yang dibutuhkan. Entity `User` saat ini hanya punya getter, jadi tambahkan method pengubah:

```
// user/User.java
public void setFullName(String fullName) {
        this.fullName = fullName;
}
```

Repository cukup ditambah method baru jika butuh query baru. Spring Data membuat query dari nama method, contohnya `findByUserId`.

### Langkah 4 — DTO

```
// profile/dto/UpdateProfileRequest.java
public record UpdateProfileRequest(
                @NotBlank(message = "Nama lengkap wajib diisi")
                @Size(max = 255, message = "Nama lengkap maksimal 255 karakter") String fullName)
{
}
```

Isi `message` di tiap anotasi dengan pesan Indonesia, karena pesan itulah yang tampil ke pengguna saat validasi gagal. Response bisa memakai ulang `UserResponse` yang sudah ada.

### Langkah 5 — Service

```
// profile/ProfileService.java
@Service
public class ProfileService {

	private final UserRepository userRepository;

	public ProfileService(UserRepository userRepository) {
		this.userRepository = userRepository;
	}

	@Transactional
	public UserResponse updateProfile(String userId, UpdateProfileRequest request) {
		User user = userRepository.findByUserId(userId)
			.orElseThrow(() -> new ApiException(ErrorCode.NOT_FOUND, "User tidak
ditemukan"));
		user.setFullName(request.fullName().trim());
		return UserResponse.from(user);
	}

}
```

Di dalam method `@Transactional`, perubahan pada entity otomatis disimpan saat method selesai, jadi tidak perlu memanggil `save()`.

### Langkah 6 — Controller

```
// profile/ProfileController.java
@RestController
@RequestMapping("/api/profile")
public class ProfileController {

	private final ProfileService profileService;

	public ProfileController(ProfileService profileService) {
		this.profileService = profileService;
	}

	@PutMapping
	public ApiResponse<UserResponse> update(@AuthenticationPrincipal Jwt jwt,
			@Valid @RequestBody UpdateProfileRequest request) {
		return ApiResponse.ok(profileService.updateProfile(jwt.getSubject(), request));
	}

}
```

`jwt.getSubject()` berisi `userId` pemilik token. Selalu ambil identitas user dari token, jangan dari body request.

### Langkah 7 — Keamanan

Semua endpoint otomatis butuh token. Hanya edit `SecurityConfig` jika endpoint baru harus bisa diakses tanpa login, dengan menambah baris `requestMatchers(...).permitAll()`.

### Langkah 8 — Jalankan dan uji

```
cd backend
./mvnw spring-boot:run          # PostgreSQL lokal (port 1035, database manote) harus menyala

# 1) login untuk mendapat token
curl -X POST localhost:8080/api/auth/login -H 'Content-Type: application/json' \
     -d '{"username":"<userid>","password":"<password>"}'

# 2) panggil endpoint baru dengan token
curl -X PUT localhost:8080/api/profile -H 'Content-Type: application/json' \
     -H 'Authorization: Bearer <accessToken>' -d '{"fullName":"Nama Baru"}'
```

---

## 4. Format response

Semua response, sukses maupun gagal, memakai satu bentuk yang sama (`common/ApiResponse`):

```
{ "status": true,  "title": "Berhasil", "code": 200, "data": { ... } }
{ "status": false, "title": "User ID/email atau password salah", "code": 4011, "data": null }
```

| Field | Isi |
|---|---|
| `status` | `true` jika berhasil, `false` jika gagal. |
| `title` | Pesan yang siap ditampilkan ke pengguna. |
| `code` | Kode HTTP standar, atau kode khusus Manote. |
| `data` | Hasilnya; `null` jika gagal atau memang tidak ada isi. |

**Sukses:** controller mengembalikan `ApiResponse.ok(data)`, `ApiResponse.created(data)` untuk data baru, atau `ApiResponse.ok()` jika tidak ada isi (misalnya setelah menghapus).

**Gagal:** service melempar `ApiException`. `GlobalExceptionHandler` mengubahnya menjadi bentuk di atas, jadi controller tidak perlu menangani error.

```
throw new ApiException(ErrorCode.NOT_FOUND, "Catatan tidak ditemukan");
throw new ApiException(ErrorCode.INVALID_CREDENTIALS);      // memakai pesan bawaan kodenya
```

**Daftar kode (`common/ErrorCode`)**

| Code | Nama | Arti |
|---|---|---|
| 200 / 201 | — | Sukses / sukses membuat data baru |
| 400 | `BAD_REQUEST` | Permintaan tidak valid (JSON rusak, ID bukan angka) |
| 4001 | `VALIDATION_FAILED` | Validasi gagal; `title` berisi pesan field pertama yang salah |
| 401 | `UNAUTHENTICATED` | Belum login (tidak mengirim token) |
| 4011 | `INVALID_CREDENTIALS` | User ID/email atau password salah |
| 4012 | `TOKEN_INVALID` | Sesi berakhir atau token tidak berlaku |
| 403 | `FORBIDDEN` | Tidak punya akses |
| 404 | `NOT_FOUND` | Data tidak ditemukan |
| 429 | `TOO_MANY_REQUESTS` | Terlalu banyak permintaan |
| 4291 | `LOGIN_LOCKED` | Login terkunci sementara |
| 500 | `INTERNAL_ERROR` | Kesalahan server; detailnya hanya ada di log |

> **Menambah kode baru:** tambahkan satu baris di `ErrorCode` dengan pola kode HTTP induk ditambah satu digit urut (misalnya 4041 untuk turunan 404), lalu tambahkan kode yang sama di `web/src/lib/api-codes.ts` dan `APICode` di iOS. Status HTTP asli tetap dikirim mengikuti kode induknya.

---

## 5. Checklist fitur backend

- ☐ Kontrak API sudah ditulis (method, URL, request, response, error).
- ☐ Migrasi baru dibuat jika skema berubah; migrasi lama tidak diedit.
- ☐ Entity dan repository diperbarui.
- ☐ DTO request punya anotasi validasi dengan `message` Indonesia; controller memakai `@Valid`.
- ☐ Controller mengembalikan `ApiResponse`; error dilempar sebagai `ApiException` dengan `ErrorCode` yang tepat.
- ☐ Logika ada di service, bukan di controller.
- ☐ Identitas user diambil dari token.
- ☐ Endpoint diuji dengan curl: kasus berhasil, input salah, dan tanpa token.